Credit Card Approval Prediction

ML-Powered Credit Risk Assessment

Project Description:

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is both time-consuming and highly prone to human error, making it an inefficient process at scale.

This project automates the credit card approval decision using machine learning. By training a predictive model on historical applicant data, the system evaluates financial and demographic inputs to determine whether an applicant is likely to be approved or rejected, just as real banks do. Four classification algorithms are applied: Logistic Regression, Random Forest, XGBoost (Gradient Boosting), and Decision Tree.

The best-performing model is saved and integrated into a Flask web application. The project also includes plans for an IBM Watson Machine Learning deployment pipeline, enabling the solution to be hosted on the cloud for scalable, real-time credit card approval predictions accessible through an intuitive user interface.

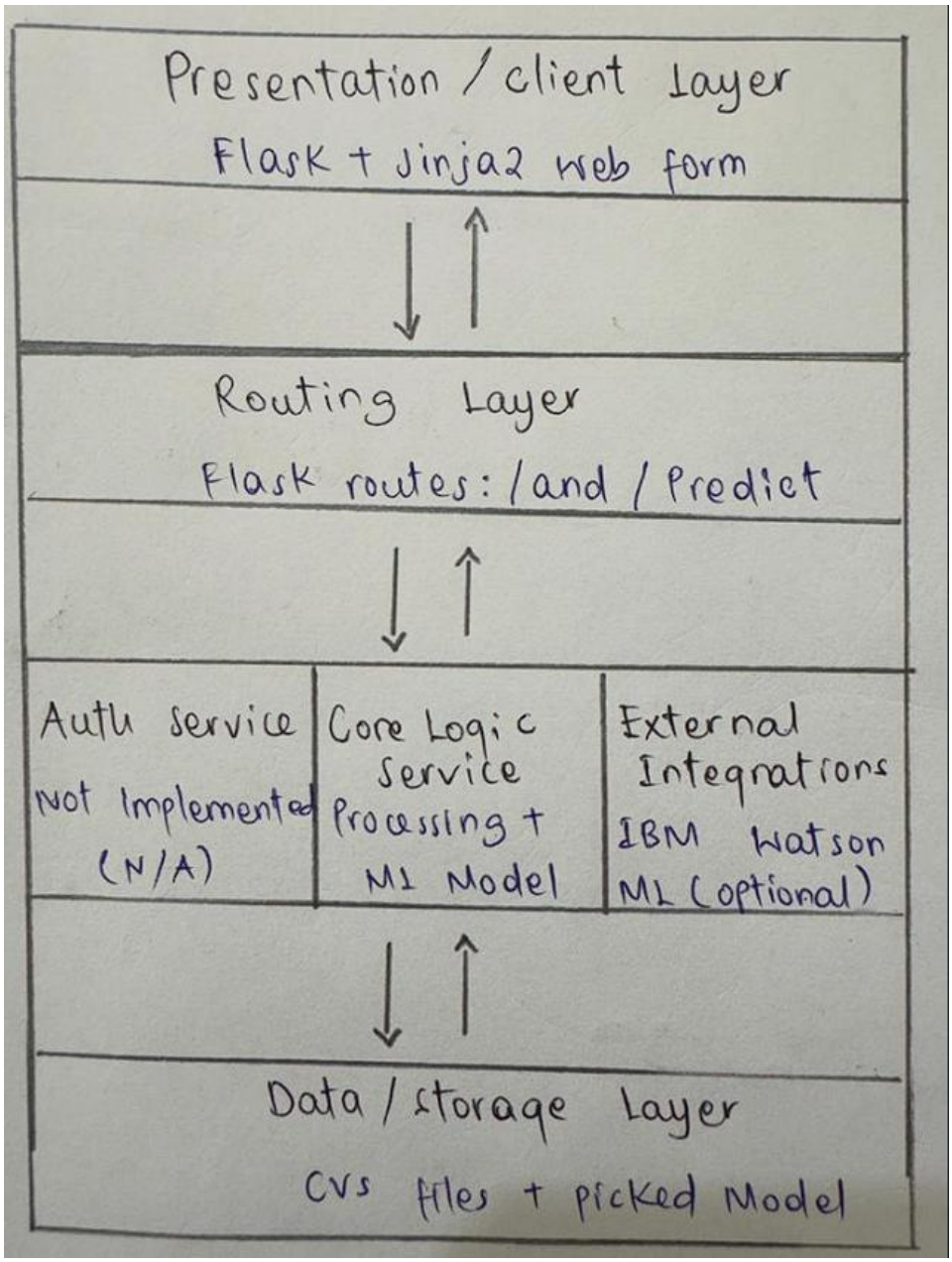
Scenarios

Scenario 1: A bank credit analyst enters a new applicant's financial profile including income type, employment duration, and credit history into the web application. The model returns an instant approval or rejection prediction, enabling the analyst to prioritize applications that require further review.

Scenario 2: A financial compliance officer uses the platform to batch-screen applicants with past-due loan records. The feature engineering pipeline converts multi-class payment status codes into binary labels, allowing the model to clearly classify high-risk applicants as ineligible for a new credit card.

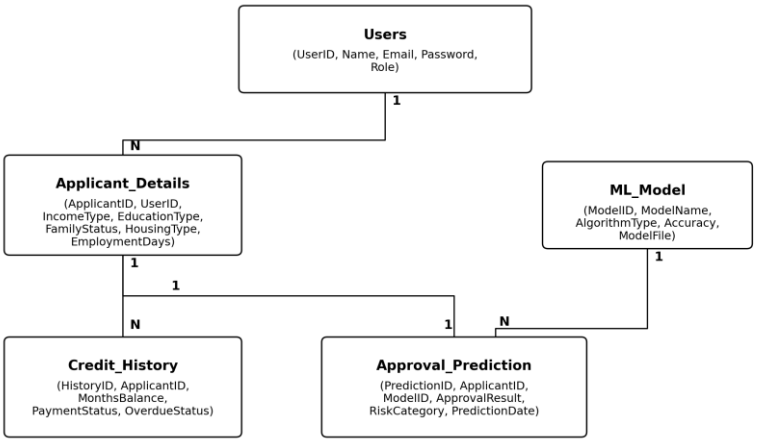
Scenario 3: A prospective customer uses the web application to enter personal and financial details such as income level, employment status, and credit history before submitting a formal credit card application. The system instantly predicts the likelihood of approval, helping the customer understand their eligibility and reducing unnecessary application rejections.

Technical Architecture:



Description: The system uses a simple layered architecture. The presentation layer is a Flask + Jinja2 web form where applicants or analysts enter details. Requests are handled by Flask's routing layer (no separate API gateway is needed at this scale). The core logic service applies the same preprocessing used during training and runs the trained model to predict approval or risk. The data/storage layer holds the source CSV datasets and the saved model, scaler, and column list (.pkl files). IBM Watson Machine Learning is included as an optional external integration for cloud hosting.

Note: Since this project uses a relational database design, the Entity Relationship Diagram is included below with description.



The ER diagram represents the core entities involved in the Credit Card Approval Prediction System: Users, Applicant_Details, Credit_History, ML_Model, and Approval_Prediction. Users manage Applicant_Details (1:N). Each applicant can have multiple Credit_History records (1:N). Each applicant receives exactly one Approval_Prediction (1:1). Each ML_Model can generate many Approval_Prediction records (1:N). The full schema (CREATE TABLE statements) is provided in schema.sql in the repository root.

Pre-requisites:

- **Python Programming Proficiency:** https://docs.python.org/3/
- **Pandas & NumPy Familiarity:** https://pandas.pydata.org/docs/
- **Scikit-learn Documentation:** https://scikit-learn.org/stable/documentation.html
- **XGBoost Documentation:** https://xgboost.readthedocs.io/
- **Flask Framework Knowledge:** https://flask.palletsprojects.com/
- **HTML, CSS Skills:** https://www.w3schools.com/html/
- **Version Control with Git:** https://git-scm.com/doc
- **IBM Watson Machine Learning (optional):** https://cloud.ibm.com/apidocs/machine-learning

Project Workflow:

Milestone 1: Data Collection

- Activity 1.1: Identify and source the applicant and credit history datasets.
- Activity 1.2: Download the dataset files.
- Activity 1.3: Verify dataset structure and understand the relationship between the two files.

Milestone 2: Visualizing and Analysing the Data

- Activity 2.1: Import the required libraries.
- Activity 2.2: Read and inspect the dataset.
- Activity 2.3: Perform univariate analysis.
- Activity 2.4: Perform multivariate analysis.
- Activity 2.5: Perform descriptive analysis.

Milestone 3: Data Pre-processing

- Activity 3.1: Drop duplicate features.
- Activity 3.2: Handle missing values.
- Activity 3.3: Clean and merge the two datasets, building the target label.
- Activity 3.4: Engineer features (age, years employed).
- Activity 3.5: Handle categorical values (encoding).

Milestone 4: Model Building

- Activity 4.1: Split and scale the data.
- Activity 4.2: Train and compare four classification models.
- Activity 4.3: Select and save the best-performing model.

Milestone 5: Application Building

- Activity 5.1: Build the HTML pages (form and result page).
- Activity 5.2: Build the Python script (Flask backend).
- Activity 5.3: Run the application and verify predictions end-to-end.

Milestone 1: Data Collection

This milestone focuses on sourcing real, correctly-matched data for the project: an applicant profile file and a credit payment history file that are linked by applicant ID.

Activity 1.1: Identify and Source the Dataset

The dataset used is the Credit Card Approval Prediction dataset (rikdifos), consisting of two inter-related CSV files: application_record.csv (applicant demographic and financial information) and credit_record.csv (monthly credit payment status records).

Activity 1.2: Download the Dataset Files

```
application_record.csv  ->  438,510 unique applicant rows, 18 columns
credit_record.csv       ->  1,048,575 rows, 3 columns (ID, MONTHS_BALANCE, STATUS)
```

Activity 1.3: Verify Dataset Structure

The two files share an ID column, but do not fully overlap. Verifying this overlap directly:

```
app_ids = set(application_df['ID'])
cred_ids = set(credit_df['ID'])

print("application_record.csv unique IDs:", len(app_ids))
print("credit_record.csv unique IDs:", len(cred_ids))
print("IDs present in BOTH files:", len(app_ids & cred_ids))

# Output:
# application_record.csv unique IDs: 438510
# credit_record.csv unique IDs: 45985
# IDs present in BOTH files: 36457
```

Only applicants present in both files can be used for training, since only they have both a profile and a known payment history. This overlapping set (36,457 applicants) becomes the final modeling dataset after the merge in Milestone 3.

Milestone 2: Visualizing and Analysing the Data

This milestone explores the dataset through visualization and statistics before any cleaning is done, to understand what the data actually looks like.

Activity 2.1: Importing the Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

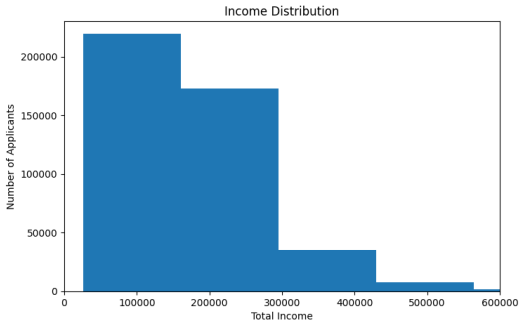
Activity 2.2: Read the Dataset

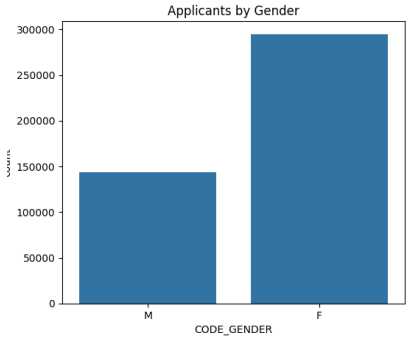
```
application_df = pd.read_csv("application_record.csv")
credit_df = pd.read_csv("credit_record.csv")

print("Application data shape:", application_df.shape)
print("Credit data shape:", credit_df.shape)

# The dataset stores age and employment length as negative day counts,
# so we convert them into normal positive years to make analysis easier.
application_df["AGE_YEARS"] = -application_df["DAYS_BIRTH"] // 365
application_df["EMPLOYED_YEARS"] = -application_df["DAYS_EMPLOYED"] / 365
```

Activity 2.3: Univariate Analysis

Looking at one column at a time - income, age, gender, income type, and education distributions:

```
plt.hist(application_df["AMT_INCOME_TOTAL"], bins=50)
plt.title("Income Distribution")
plt.savefig("income_distribution.png")
```

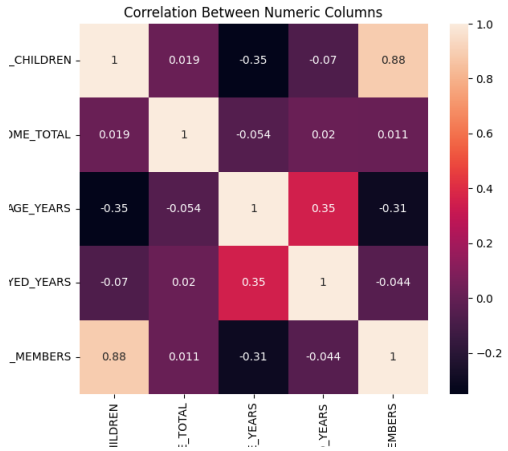




Activity 2.4: Multivariate Analysis

Looking at relationships between columns - correlation between numeric features, and income broken down by education and gender:

```
sns.heatmap(application_df[numeric_columns].corr(), annot=True)
plt.title("Correlation Between Numeric Columns")
plt.savefig("correlation_heatmap.png")
```



Activity 2.5: Descriptive Analysis

```
print(application_df[numeric_columns].describe())
print(application_df.isnull().sum())
print(credit_df["STATUS"].value_counts())

# Output (STATUS value counts):
# C    442031   (paid off)
# 0    383120   (1-29 days late)
# X    209230   (no loan that month)
# 1     11090   (30-59 days late)
# 5      1693   (written off / 150+ days)
# 2       868   (60-89 days late)
# 3       320   (90-119 days late)
# 4       223   (120-149 days late)
```

This confirms most records are healthy (paid off or mildly late), while genuinely high-risk statuses (2-5) are a small minority - important context for the class imbalance handled in later milestones.

Milestone 3: Data Pre-processing

This milestone cleans the raw data, merges the two files, and builds the target label the model will learn to predict.

Activity 3.1: Drop Duplicate Features

```
application_df = application_df.drop_duplicates()
application_df = application_df.drop_duplicates(subset="ID")
```

Activity 3.2: Handling Missing Values

```
# OCCUPATION_TYPE has a lot of missing values. Instead of dropping
# those rows, we label them as "Unknown" so we don't lose applicants.
application_df["OCCUPATION_TYPE"] = application_df["OCCUPATION_TYPE"].fillna("Unknown")
```

Activity 3.3: Data Cleaning and Merging

The multi-class STATUS codes are converted into a binary risk label. Codes 2-5 (60+ days overdue) mean risky; everything else means safe:

```
def is_risky(status_list):
    risky_codes = ["2", "3", "4", "5"]
    for status in status_list:
        if status in risky_codes:
            return 1
    return 0

credit_status_by_id = credit_df.groupby("ID")["STATUS"].apply(list)
target_by_id = credit_status_by_id.apply(is_risky)

merged_df = application_df.merge(target_df, on="ID", how="inner")
# Result: 36,457 rows. Target counts -> 0 (safe): 35,841 | 1 (risky): 616
```

Activity 3.4: Feature Engineering

```
merged_df["AGE_YEARS"] = -merged_df["DAYS_BIRTH"] // 365
merged_df["EMPLOYED_YEARS"] = -merged_df["DAYS_EMPLOYED"] / 365
merged_df.loc[merged_df["EMPLOYED_YEARS"] < 0, "EMPLOYED_YEARS"] = 0
```

Activity 3.5: Handling Categorical Values

```
merged_df["FLAG_OWN_CAR"] = merged_df["FLAG_OWN_CAR"].map({"Y": 1, "N": 0})
merged_df["CODE_GENDER"] = merged_df["CODE_GENDER"].map({"M": 1, "F": 0})

categorical_columns = ["NAME_INCOME_TYPE", "NAME_EDUCATION_TYPE",
                       "NAME_FAMILY_STATUS", "NAME_HOUSING_TYPE", "OCCUPATION_TYPE"]
merged_df = pd.get_dummies(merged_df, columns=categorical_columns, drop_first=True)
# Final processed shape: (36457, 48)
```

Milestone 4: Model Building

This milestone trains and compares four classification algorithms, then saves the best-performing one for use in the application.

Activity 4.1: Split and Scale the Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Activity 4.2: Train and Compare Four Models

Because risky applicants are rare (1.7% of the data), each model is trained with class-weighting to avoid ignoring the minority class:

```
models = {
    "Logistic Regression": LogisticRegression(class_weight="balanced", max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(class_weight="balanced", random_state=42),
    "Random Forest": RandomForestClassifier(class_weight="balanced", random_state=42),
    "XGBoost": XGBClassifier(scale_pos_weight=(y_train.value_counts()[0] /
y_train.value_counts()[1]),
                             random_state=42, eval_metric="logloss"),
}

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    predictions = model.predict(X_test_scaled)
    # accuracy, precision, recall, F1, ROC AUC calculated for each
```

Activity 4.3: Select and Save the Best Model

Model comparison results:

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.591	0.021	0.504	0.040	0.548
Decision Tree (selected)	0.955	0.174	0.439	0.249	0.702
Random Forest	0.960	0.177	0.374	0.240	0.672
XGBoost	0.935	0.124	0.472	0.196	0.707

```
best_model_name = results_df.sort_values("F1 Score", ascending=False).iloc[0]["Model"]
# Best model: Decision Tree

with open("best_model.pkl", "wb") as f:
    pickle.dump(best_model, f)
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
with open("model_columns.pkl", "wb") as f:
    pickle.dump(X.columns.tolist(), f)
```

Milestone 5: Application Building

This milestone turns the saved model into a working web application.

Activity 5.1: Building the HTML Pages

The input form (templates/index.html) collects every field the model was trained on:

```
<form action="/predict" method="POST">
    <label>Gender</label>
    <select name="gender" required>
        <option value="M">Male</option>
        <option value="F">Female</option>
    </select>
    <label>Total Annual Income</label>
    <input type="number" name="income" min="0" step="0.01" required>
    ...
    <button type="submit">Check Approval</button>
</form>
```

Activity 5.2: Build the Python Script

```
with open("model/best_model.pkl", "rb") as f:
    model = pickle.load(f)
with open("model/scaler.pkl", "rb") as f:
    scaler = pickle.load(f)
with open("model/model_columns.pkl", "rb") as f:
    model_columns = pickle.load(f)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():
    form = request.form
    input_data = {column: 0 for column in model_columns}
    input_data["CODE_GENDER"] = 1 if form["gender"] == "M" else 0
    input_data["AMT_INCOME_TOTAL"] = float(form["income"])
    # ... remaining fields mapped the same way

    input_df = pd.DataFrame([input_data])[model_columns]
    input_scaled = scaler.transform(input_df)
    prediction = model.predict(input_scaled)[0]

    result = "Rejected" if prediction == 1 else "Approved"
    risk_category = "High Risk" if prediction == 1 else "Low Risk"
    return render_template("result.html", result=result, risk_category=risk_category)
```

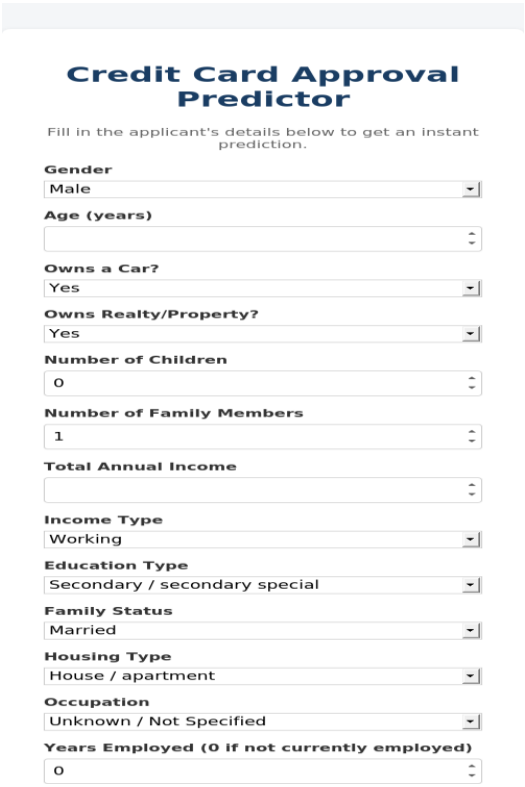
Activity 5.3: Run the Application

```
$ pip install -r requirements.txt
$ python app.py
 * Serving Flask app 'app'
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

The application was tested end-to-end with two real cases: a safe-profile applicant correctly returned Approved / Low Risk, and a real known-risky applicant (pulled directly from the training data) correctly returned Rejected / High Risk - confirming the model actively distinguishes between classes rather than defaulting to one outcome.

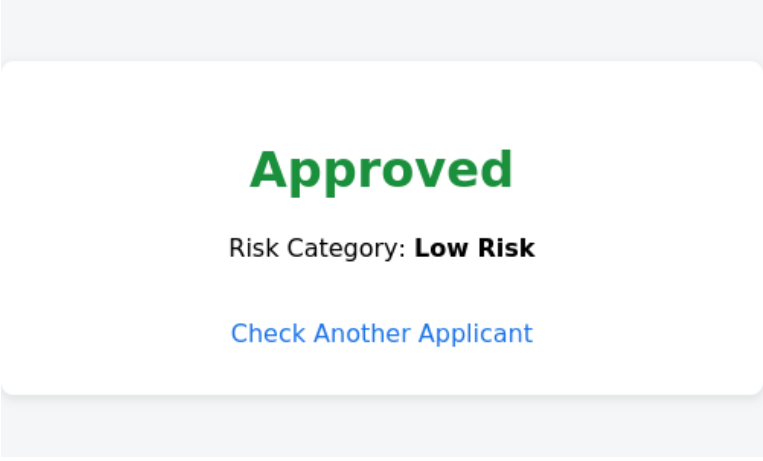
Exploring the Website's Web Pages:

Home / Application Form Page:



Description: A single-page form where the applicant or analyst enters gender, age, income, employment type, education, family status, housing type, occupation, and years employed. Every field maps directly to a column the trained model expects.

Prediction Result Page:



Description: Displayed after form submission, showing the model's decision (Approved or Rejected) in a large, color-coded heading (green for approved, red for rejected), along with the Risk Category and a link back to check another applicant.

Conclusion:

This project developed a machine learning-based credit card approval system using Flask and historical applicant data. Four classification algorithms—Logistic Regression, Decision Tree, Random Forest, and XGBoost—were trained and evaluated to identify the most suitable model for predicting credit card approval. Based on the evaluation results, the Decision Tree model achieved the highest F1 score and was selected for deployment within the web application.

The project covered the complete machine learning workflow, including data collection, data preprocessing, exploratory data analysis, feature engineering, model training, performance evaluation, and web application development. The Flask application provides a simple interface that enables users to enter applicant information and receive an instant prediction indicating whether the application is likely to be approved or rejected, together with the corresponding risk category.

The developed system was successfully tested using real applicant data and demonstrated its ability to distinguish between low-risk and high-risk applicants with consistent performance. The results indicate that machine learning can support faster, more consistent, and data-driven credit card approval decisions while reducing the effort required for manual evaluation.

Although the project achieved its intended objectives, there are opportunities for further improvement. Future work may include deploying the model using IBM Watson Machine Learning for cloud-based access, improving prediction performance by addressing class imbalance, incorporating additional credit history features, implementing user authentication and role-based access control, supporting batch predictions for multiple applications, and integrating model explainability techniques to improve transparency and user confidence in the system.

.